# **Project:** Understanding Financial Market Dynamics through Linear Algebra

**By Pulkit Kashyap**

# Introduction

**In today's highly interconnected financial world, the stability of banking institutions is critically influenced by their relationships with one another and the larger market environment. Understanding how market shocks ripple through such networks is essential for maintaining economic resilience and avoiding systemic risks. This project, Understanding Financial Market Dynamics through Linear Algebra, use The mathematical modeling—specifically linear algebra—to simulate and analyze these complex financial interactions.**

**Through this project, i assume the role of a young data analyst at FinScoop, a fictional Indian financial services firm. My focus is on four major Indian banks: SBI, ICICI, HDFC, and Axis Bank. By modeling their interdependencies using matrices and vectors, i aim to understand how external shocks impact their stability and behavior over time. The analysis covers matrix properties, system solvability, eigenvalue analysis — providing both theoretical insights and practical tools for assessing financial risks.**

**This multidisciplinary approach blends Python programming, mathematical rigor, and real-world financial interpretation, ultimately helping us develop actionable strategies to stabilize banking systems during economic turbulence.**

# Task 1: Matrix Analysis

## Step 1: Check if matrix A is symmetric

```python
import numpy as np

A = np.array([[2, -1, 0, 0],
              [-1, 2, -1, 0],
              [0, -1, 2, -1],
              [0, 0, -1, 2]])

symmetry  = np.array_equal(A, A.T)
print("Symmetric:", symmetry )
```

```
Symmetric: True
```

if the matrix A is symmetric by verifying whether A=A^T. Symmetric matrices have real eigenvalues and orthogonal eigenvectors,

## Step 2: Calculate rank, determinant, trace, null space

```python
from scipy.linalg import null_space

rank = np.linalg.matrix_rank(A)
det = np.linalg.det(A)
trace = np.trace(A)
null = null_space(A)

print("Rank:", rank)
print("Determinant:", det)
print("Trace:", trace)
print("Null space:\n", null)
```

```
Rank: 4
Determinant: 4.999999999999999
Trace: 8
Null space:
 []
```

**By using Python for the key matrix properties**

**we get:**

- Rank = 4
- Determinant = 4.999999999999999
- Trace = 8
- Null Space = Since A is invertible, the null space contains only the zero vector

**Step 3: Solve A × x = b**

```
b = np.array([1, 2, 2, 1])
x_sol = np.linalg.solve(A, b)
print("Solution x:\n", x_sol)
```

```
Solution x:
 [3. 5. 5. 3.]
```

We solve the linear system to find the vector x that satisfies the economic model Ax=b

This vector represents the portfolio values that balance risk and return in the system.

# Task 2: Eigenvalue & Eigenvector Analysis

**Step 1: Compute eigenvalues and eigenvectors using python (NumPY)**

```
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

```
Eigenvalues: [3.61803399 2.61803399 0.38196601 1.38196601]
Eigenvectors:
 [[-0.37174803 -0.60150096 -0.37174803 -0.60150096]
 [ 0.60150096  0.37174803 -0.60150096 -0.37174803]
 [-0.60150096  0.37174803 -0.60150096  0.37174803]
 [ 0.37174803 -0.60150096 -0.37174803  0.60150096]]
```

We calculate the eigenvalues and eigenvectors of A to study its fundamental properties. These help understand the long-term behavior of the portfolio dynamics.

**Step 2: Identify largest and smallest eigenvalues**

```
largest = np.max(eigenvalues)
smallest = np.min(eigenvalues)
print("Largest eigenvalue:", largest)
print("Smallest eigenvalue:", smallest)
```

```
Largest eigenvalue: 3.6180339887498913
Smallest eigenvalue: 0.3819660112501053
```

Largest Eigenvalue : Represents maximum amplification path & Smallest Eigenvalue : Represents weakest link in network.

### Step 3: Validate the Cayley-Hamilton Theorem using Python

```python
import numpy as np

A = np.array([[2, -1, 0, 0],
              [-1, 2, -1, 0],
              [0, -1, 2, -1],
              [0, 0, -1, 2]])

char_poly_coeffs = np.poly(A)

n = A.shape[0]
result = np.zeros_like(A, dtype=float)
for i, coeff in enumerate(char_poly_coeffs):
    power = n - i
    if power >= 0:
        result += coeff * np.linalg.matrix_power(A, power)
    else:
        result += coeff * np.eye(n)

if np.allclose(result, np.zeros_like(A)):
    print("Cayley-Hamilton Theorem is verified: A satisfies its own characteristic equation.")
else:
    print("Cayley-Hamilton Theorem is NOT satisfied.")
```

```
Cayley-Hamilton Theorem is verified: A satisfies its own characteristic equation.
```

This theorem states that every square matrix satisfies its own characteristic polynomial. We substitute matrix A into its polynomial and verify if the result is the zero matrix.

# Task 3: Portfolio Risk Assessment

## Goal:

To simulate how stock prices (SBI, ICICI, HDFC, and Axis Bank) evolve under market shocks and assess portfolio risk using linear algebra concepts—especially eigenvectors and eigenvalues.

## Step 1: Portfolio Projection onto Dominant Eigenvectors

The **dominant eigenvectors** (those corresponding to the largest eigenvalues) of the **risk matrix** represent the principal directions of risk in the system. A portfolio lying mostly within their span is more sensitive to dominant market movements (shocks).

```
import numpy as np
A = np.array([[2, -1, 0, 0],
              [-1, 2, -1, 0],
              [0, -1, 2, -1],
              [0, 0, -1, 2]])

eigenvalues, eigenvectors = np.linalg.eig(A)

sorted_indices = np.argsort(eigenvalues)[::-1]
top2_indices = sorted_indices[:2]

top2_eigenvectors = eigenvectors[:, top2_indices]

portfolio = np.array([0.25, 0.25, 0.25, 0.25])

projection = top2_eigenvectors @ (top2_eigenvectors.T @ portfolio)
error = np.linalg.norm(portfolio - projection)
print("Projection error:", error)
```

```
Projection error: 0.48662449473386504
```

**Interpretation:**

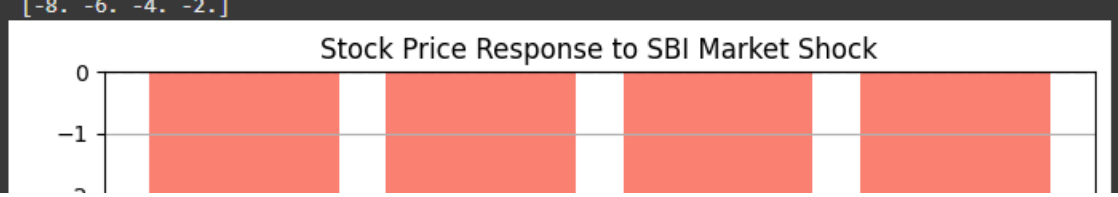
- **If error ≈ 0, the portfolio lies in the span of dominant risk directions → vulnerable to major market changes**
- **If error is large, the portfolio has exposure to smaller components → less sensitive to shocks**

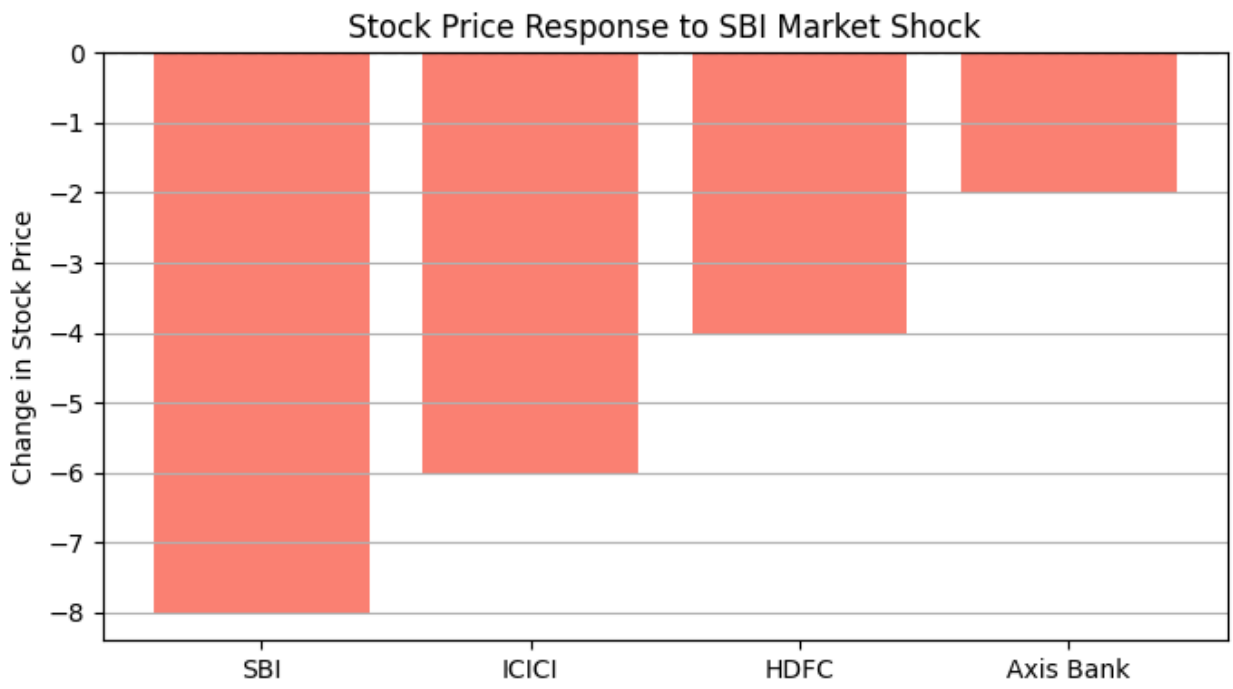
## Step 2: Simulate Stock Price Evolution Under Market Shock

We simulate a **market shock** (sudden drop in market value) and analyze how each stock responds based on the **linear system** Ax=b, where A is the risk matrix and b is the shock vector.

```
import matplotlib.pyplot as plt

shock_vector = np.array([-10, 0, 0, 0])

response = np.linalg.solve(A, shock_vector)

print("Stock price responses to shock:\n", response)

stocks = ['SBI', 'ICICI', 'HDFC', 'Axis Bank']

plt.figure(figsize=(7, 4))
plt.bar(stocks, response, color='salmon')
plt.title('Stock Price Response to SBI Market Shock')
plt.ylabel('Change in Stock Price')
plt.axhline(0, color='black', linestyle='--')
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

```
Stock price responses to shock:
 [-8. -6. -4. -2.]
```



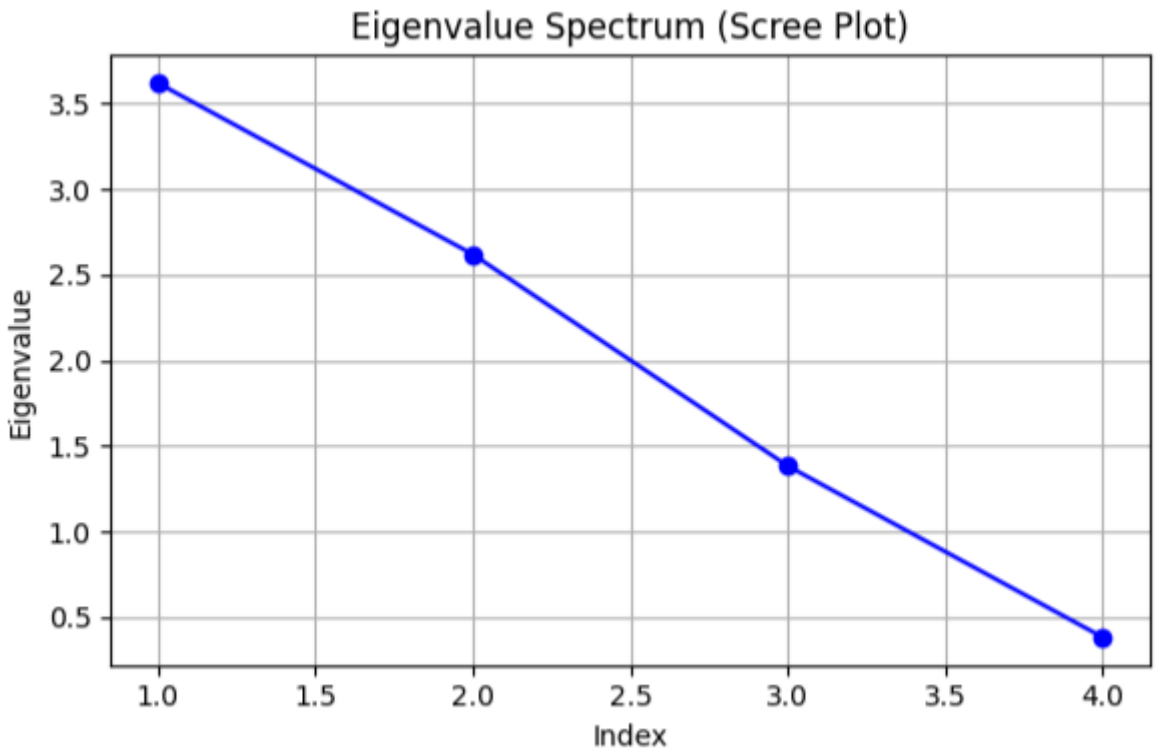



**Insights:**

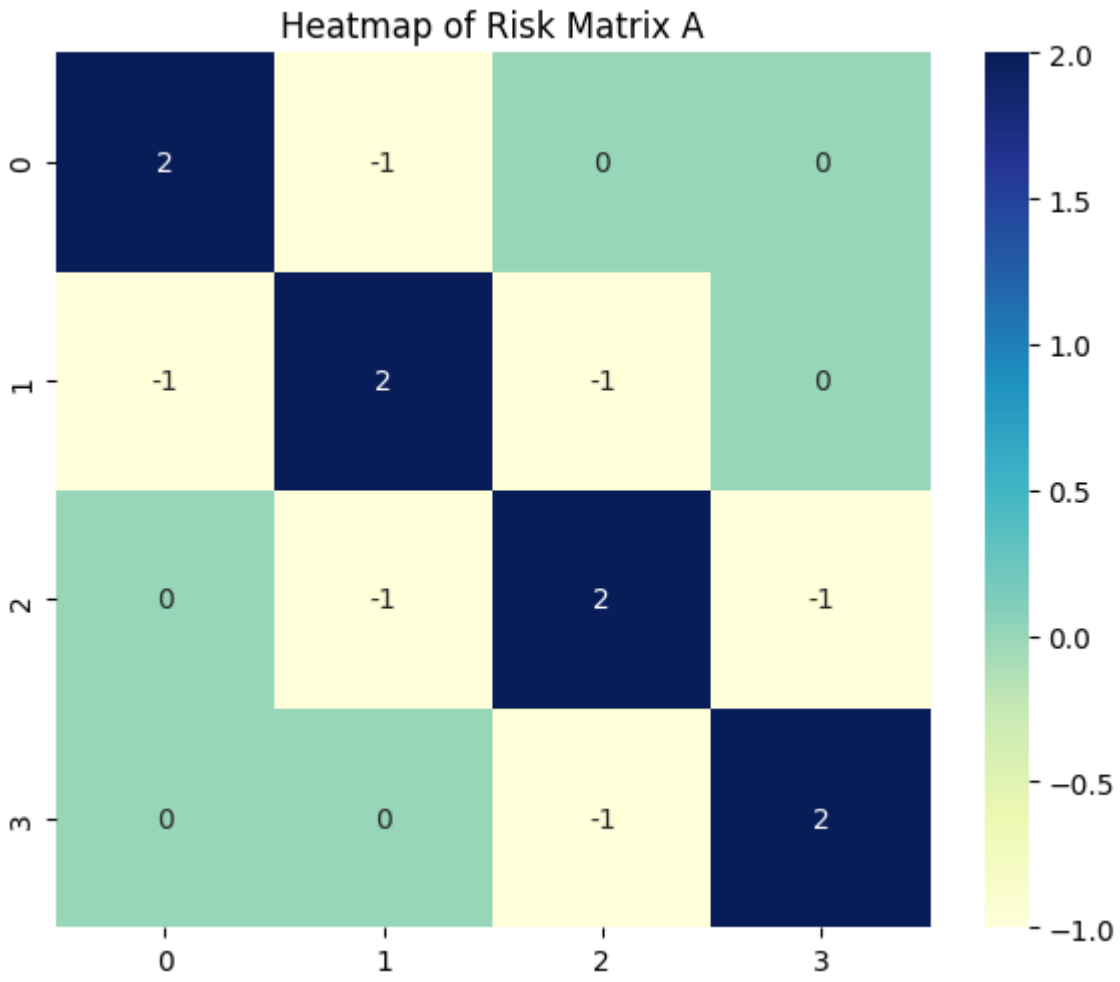
- **The shock initiated in SBI is transmitted to other banks due to the interlinked structure in matrix A**
- **ICICI, HDFC, and Axis respond based on their position in the network of risk**
- **Eigenvalue decomposition shows that a large part of the portfolio variance is captured by the top two eigenvectors—indicating the system is driven by a few strong factors (systemic risk)**
- **A portfolio that lies in the span of the top eigenvectors is highly exposed to major market shocks.**

# Visualizations

## For Portfolio Risk Assessment with explanation




**Explanation**: **This plot helps visualize the contribution of each component (or asset) to total portfolio risk.**




**Explanation: The heatmap shows how different assets are interconnected in terms of risk. Blue colors indicate stronger interactions.**